RCP Analysis Codebase Documentation

This document provides an overview of the Python scripts, functions, classes, and Jupyter notebooks in the `RCP_analysis` repository.

Table of Contents

---

1. Batch Scripts (`batch_scripts/`)**

These scripts are typically used to run batch analysis or alignment pipelines across datasets.

`NPRW_Intan_analysis_mf.py`

Path: batch_scripts/NPRW_Intan_analysis_mf.py

Description: Preprocesses Intan neural data recorded from Neuropixels-like linear silicon probes (NPRW). Runs artifact detection/removal, spike detection using matched filters (MF), and calculates continuous firing rates.

Functions:

- `main()`

`UA_BR_analysis_mf.py`

Path: batch_scripts/UA_BR_analysis_mf.py

Description: Performs artifact correction and spike extraction on Utah Array (UA) Blackrock recordings using Incremental Principal Component Analysis (IPCA) to learn spatial templates, followed by matched-filter spike detection.

Functions:

- `detect_artifacts_matched_filter(trace, template, fs, expected_times, search_radius_ms, threshold_std)`
  Description: Detect artifact times using matched filtering with a learned template. Parameters

trace : 1D array The signal to search (in µV) template : 1D array The normalized artifact template fs : float Sampling frequency in Hz expected_times : array or None Expected artifact sample indices (from Intan timing). If provided, detection is refined around these times. If None, finds all peaks. search_radius_ms : float Search radius around expected times (ms) threshold_std : float Detection threshold in standard deviations above baseline correlation Returns

detected_times : array Sample indices where artifacts were detected correlation : array The matched filter output (for debugging)

- `_find_first_significant_peak(tr_local, ref_ratio)` *Description*: Given a local trace fragment (already roughly centred on a peak), find the first peak in the absolute derivative that exceeds `ref_ratio` of the local maximum derivative. Returns the index relative to the start of `tr_local`.
- `load_anchor_for_session(out_base, session)` *Description*: Return (anchor_sample_intan, fs_intan) from figures/align_BR_to_Intan/br_to_intan_shifts.csv. Falls back to deriving anchor_sample from anchor_ms if needed.
- `main()`

Classes:

- `class IPCACorrectedRecordingSegment` *Methods*:
  - `__init__(self, parent_recording_segment, micro_map, micro_corrected)`
  - `get_num_samples(self)`
  - `get_traces(self, start_frame, end_frame, channel_indices)`
- `class IPCACorrectedRecording` *Methods*:
  - `__init__(self, parent_recording, micro_map, micro_corrected)`

align_VOG_to_br.py

Path: batch_scripts/align_VOG_to_br.py

Description: Aligns video oculography (VOG) eye-tracking CSV data to the Blackrock timeline by finding sync pulses (e.g. triangle wave or rising edges) in both datasets.

Functions:

- `main()`

align_dlc_two_cams_to_br.py

Path: batch_scripts/align_dlc_two_cams_to_br.py

Description: Aligns 2D keypoints tracked by DeepLabCut (DLC) from two cameras with the Blackrock sync timescale using rising-edge frame triggers recorded on auxiliary channels. It filters low-likelihood points and resamples kinematics.

Functions:

- `filter_low_likelihood(df, threshold)` *Description*: Iterate columns to find _likelihood cols. If col < threshold, set corresponding _x and _y to NaN.
- `main()`

analyze_lfp_bands.py

Path: batch_scripts/analyze_lfp_bands.py

Description: Processes Utah array LFP bands (alpha, beta, gamma, etc.) around stimulation events. Includes IPCA artifact correction to prevent OOM errors, zero-phase filtering, spectral detrending (1/f subtraction), and aggregates baseline epochs.

Functions:

- `_find_first_significant_peak(tr_local, ref_ratio)` *Description*: Given a local trace fragment (already roughly centred on a peak), find the first peak in the absolute derivative that exceeds `ref_ratio` of the local maximum derivative. Returns the index relative to the start of `tr_local`.
- `_get_electrode_region(elec_id)` *Description*: Get region name for a Utah Array electrode based on 1-based ID. Standard mapping:
  - 1-64 = SMA
  - 65-128 = PMd
  - 129-192 = M1i
  - 193-256 = M1s
- `_get_stim_frequency_from_metadata(br_idx)` *Description*: Get stimulation frequency from metadata CSV.
- `_get_ua_port_from_metadata(br_idx)` *Description*: Get UA port (A or B) from metadata CSV.
- `find_peristim_files(sess_name, br_idx)`
- `_normalize_metadata(df_raw)` *Description*: Normalize column names to lowercase with underscores.
- `_find_baseline_groups(df_norm)` *Description*: Return mapping (ua_port_norm, depth_mm_norm) -> row indices of baseline rows. Mirrors the grouping logic from extract_peri_IR_and_concat_baseline.py.
- `apply_1f_detrending_chunked(epochs, rel_t_epoch, win_base, fs, chunk_size, plot_debug, debug_out_dir)` *Description*: Applies adaptive 1/f spectral subtraction in the time-domain using a baseline period. Processes channels in chunks to prevent OOM. Args: epochs (np.ndarray): (n_trials, n_ch, n_time) rel_t_epoch (np.ndarray): Time axis in ms win_base (tuple): Baseline window (start_ms, end_ms) for 1/f fitting fs (float): Sampling rate (Hz) chunk_size (int): Channels per chunk Returns: np.ndarray: 1/f detrended epochs
- `filter_zero_phase(epochs, fs, low, high, order, chunk_size)` *Description*: Apply zero-phase bandpass filter using filtfilt along axis 2 (time). Processes in chunks along the channel axis (axis=1) to prevent massive memory OOM spikes when filtering long continuous recordings. Args: epochs (np.ndarray): (n_trials, n_ch, n_time) fs (float): Sampling rate (Hz) low (float): Low cutoff (Hz) high (float): High cutoff (Hz) order (int): Filter order chunk_size (int): Number of channels to filter at once Returns: np.ndarray: Filtered data
- `apply_ipca_correction(rec_ns6, starts_ua, ends_ua, br_idx)` *Description*: Applies IPCA artifact removal to stimulation windows, using cross-channel templates learned individually per brain region. This function matches the logic in UA_BR_analysis_mf.py exactly.
- `build_lfp_preprocessing_pipeline(recording, stim_indices_native, curr_sess_name, local_radii)` *Description*: Custom pipeline with purely Bandpass and Resampling operations. Blanking

/ CMR are bypassed heavily for Utah IPCA.
- `extract_single_epoch(rec_obj, center_idx, pre_samps, post_samps)` *Description*: Extract a single epoch from a recording object. Args: rec_obj: SpikeInterface recording object center_idx: Center sample index (typically stim onset) pre_samps: Samples before center post_samps: Samples after center Returns: np.ndarray of shape (n_ch, n_time) or None if out of bounds
- `slice_epoch(data, t_axis, target_win)` *Description*: Slice epoch data to a specific time window. Args: data: np.ndarray of shape (n_trials, n_ch, n_time) t_axis: Time axis array target_win: Tuple of (start_ms, end_ms) Returns: Tuple of (sliced_data, sliced_time_axis)
- `_apply_native_ua_mapping(rec_raw, UA_MAP, br_idx, metadata_csv)`
- `process_baseline_group_utah(group_key, session_infos)` *Description*: Process multiple baseline sessions for the same Port/Depth, aggregating epochs. Optimized to load each recording only once for all targets (A, B, None).
- `process_utah_session(sess_name, br_idx, shift_ms, fs_intan, shift_sample)`
- `quick_check_stim(sess_path, stream_name, intan_idx, br_idx)` *Description*: Check if stim file or aligned file exists and has events.
- `main()`

Classes:

- `class IPCACorrectedRecordingSegment` *Methods*:
  - `__init__(self, parent_recording_segment, micro_map, micro_corrected)` *Description*: micro_map: list of tuples (p_start, p_end) indicating where artifacts are micro_corrected: np.ndarray of shape (n_events, n_time, n_channels) holding the IPCA patches
  - `get_num_samples(self)`
  - `get_traces(self, start_frame, end_frame, channel_indices)`
- `class IPCACorrectedRecording` *Description*: A SpikeInterface BaseRecording that lazily streams disk data and surgically inserts pre-computed IPCA artifact patches on the fly. *Methods*:
  - `__init__(self, parent_recording, micro_map, micro_corrected)`

compute_br_to_intan_shifts.py

Path: batch_scripts/compute_br_to_intan_shifts.py

Description: Aligns Blackrock and Intan recordings by cross-correlating sync pulse templates recorded on both systems, outputting shifts (in samples and milliseconds) to a CSV file.

Functions:

- `_xcorr_normalized(x, y)`
- `load_template(template_mat_path)`
- `find_locs_via_template(adc_lock, template, fs, peak)`
- `refine_shift_with_loc_and_br0(rec_br, br_ch, fs_br, adc_triangle_intan, fs_intan, loc, search_n_br, br_window_sec, normalize_both)` *Description*: BR origin is 0. Align BR[0:W) against Intan starting at time loc/fs_intan, then refine with additional ±n BR-sample shifts. Return: shift_sample (Intan), delta_intan_samples, dt_ms, debug dict.
- `main()`

extract_peri_stim.py

Path: batch_scripts/extract_peri_stim.py

Description: Slices aligned multi-modal data (neural firing rates, LFP, 3D kinematics, touchscreen events) into trial epochs around stimulation/event triggers, computing PSTH medians, variances, and trial-level segments.

Functions:

- `_get_valid_events(event_ms, rec_time, win_ms)` *Description*: Return mask over event_ms where [event+win0, event+win1] is in recording time.
- `_as_list(x)`
- `_last(x)`
- `_median_behavior_line(series_on_common, t_common_ms, event_ms, win_ms, baseline_ms, min_trials)` *Description*: Return: line : (T,) median peri-stim trace across kept trials rel_t: (T,) relative time axis n_kept: int, number of kept trials segs: (n_kept, T) baseline-zeroed single-trial segments
- `_median_lines_for_columns(series_on_common, t_common_ms, event_ms)` *Description*: Return: lines : (D, T) median line per column rel_t_out : (T,) relative time axis n_trials : int, max number of kept trials across columns segs_all : (n_trials, D, T) baseline-zeroed segments per column, padded with NaNs where a column has fewer trials
- `_ordered_xy_indices(cam_cols, keypoints)`
- `_select_matrix(cam, cols, keypoints)`
- `_z_per_column(M)`
- `_parse_delay_ms(val)`
- `_fmt_num(x)`
- `_interp_nans_1d_gentle(x, max_gap)` *Description*: Fill only interior NaN runs up to max_gap by linear interp. Leading/trailing NaNs stay NaN; large gaps stay NaN.
- `_interp_nans_2d_by_col(arr, max_gap)` *Description*: Apply _interp_nans_1d_gentle to each column of a 2D array.
- `_read_csv_robust(path)` *Description*: Try several common encodings and tolerate a few bad rows.
- `build_title_from_csv(csv_path)`
- `_compute_trial_labels(event_ms, t_ms, ts_char, ts_num, win_ms)`
- `_behavior_medians_for_label(cam_z, behv_t, event_ms, labels, target_label)` *Description*: Recompute behavior medians using only stims with labels==target_label.
- `_behavior_valid_mask_from_cam0(cam0_z, behv_t, event_ms, win_ms, baseline_ms, min_trials)` *Description*: Decide which stim events have 'good' behavior traces based on cam0_z. Steps:
    1. Use RCP_analysis.extract_peristim_segments to find the subset of stims that produce valid, full-length windows on the behavior timebase.
    2. Among those, keep only events whose baseline window has at least one finite value.
    3. Map that decision back to a boolean mask over the original stim_ms array. Returns: mask : (n_stims,) bool, True where behavior is usable.
- `_save_peristim()` *Description*: Save one .npz and/or .mat
    - Target subset (A/B) or
    - all events (target_name=None), e.g. AtRest
- `extract_one_file(aligned_path, out_dir, use_ir_ms, split_targets, is_control)` *Description*: Load aligned .npz, compute peri-stim med/var for NPRW+UA,
    - behavior/VOG/ts_state peri-stim summaries, and save to PERI_ROOT.
- `main()`

make_aligned_npz_and_mat.py

Path: batch_scripts/make_aligned_npz_and_mat.py

Description: Synchronizes all neural recordings, DLC tracking, touchscreen logs, and oculography into unified, resampled `.npz` and `.mat` format files for MATLAB/Python downstream analysis.

Functions:

- **`_find_aligned_DLC_for_br_idx(behv_root, br_idx)`**
  Description: Find the newest aligned DLC CSV for the given BR index, with preference: both_cams → Cam-0 → Cam-1. Returns

  (path, kind) path : Path | None kind : 'both' | 'cam0' | 'cam1' | 'none' Matches filenames like: *..._both_cams_aligned.csnv ...Cam-0_aligned.csv* ..._Cam-1_aligned.csv
- `_parse_SI_peaks(peaks, n_channels)`
- `_ua_perm_by_region(ua_region, ua_elec, ua_nsp, region_order, within)`
- `parse_bool(x)`
- `_to_mat(x)` *Description*: Recursively convert a value to something savemat can handle.
- `main()`

plot_bin_counts_per_target.py

Path: batch_scripts/plot_bin_counts_per_target.py

Description: Plots and tabulates trial counts, stimulation conditions, and success rates across different targets (e.g., target A vs B) to inspect experimental session yield.

Functions:

- `_port_from_baseline_path(npz_path)` *Description*: Extract port 'A' or 'B' from the baseline filename / path. Looks for 'port_A' / 'port B' / 'portA' etc.
- `_build_impedance_dict_for_target()` *Description*: Load UA impedances for both ports A and B from the TextEdit dumps. Returns (imp_A, imp_B) where each is {elec#: impedance_kΩ}.
- `_unit_to_kohm(val, unit)`
- `_read_text_loose(p)`
- `load_impedances_from_textedit_dump(path_like)` *Description*: Parse lines like 'elec1-5 201 kOhm' from a loose text dump. Returns { Elec#: impedance_kΩ }.
- `_simple_beh_labels(names, keypoints)` *Description*: Map raw DLC-style names like 'DLC_Resnet50_..._wrist_x' -> 'wrist_x' 'cam0_elbow_y' -> 'elbow_y' If a keypoint/axis can't be found, fall back to the original name.
- `_strip_cam_prefix(n)` *Description*: Turn "cam0_wrist_x" -> "wrist_x", "cam1_middle_finger_tip_y" -> "middle_finger_tip_y"
- `_get_first_array(z, names)` *Description*: Try a list of possible field names; return first present.
- `_get_int(z, names, default)`
- `_plot_one_baseline_npz(npz_path, target_label, ua_imp)` *Description*: Load a baseline__*.npz from PeriStim/Target_X and plot median + var figures into PeriStim/Target_X/figures using stacked_heatmaps_plus_behv.

- `_plot_one_peristim_npz(npz_path, target_label, ua_imp_a, ua_imp_b)` *Description*: Load a peristim__*.npz from PeriStim/Target_X and plot median + var figures into PeriStim/Target_X/figures.
- `main()` *Description*: For each of PeriStim/Target_A and PeriStim/Target_B:
  - plot all baseline__*.npz
  - plot all peristim__*.npz Figures go into PeriStim/Target_X/figures.

plot_complete_shaded_BT.py

Path: batch_scripts/plot_complete_shaded_BT.py

Description: Generates comprehensive peri-stimulus firing rate heatmaps for Utah Array regions and Neuropixels rows, overlaid with shaded kinematic trajectories (position/velocity) for specific trial conditions.

Functions:

- `_compute_target_means_for_cam(cam_pos_segs, cam_names, cam_rel_t, ts_state_segs, ts_state_rel_t, target_suffix)` *Description*: Uses ts_state == 1 as the 'on' state and resamples ts_state onto cam_rel_t using nearest-neighbor in time. Returns

  idx_mask : (n_kps,) bool mean_xy : (2,) global mean [x, y] when ts_state==1 mean_xy_per_trial : (n_trials, 2) per-trial mean [x, y] when ts_state==1
- `_strip_cam_prefix(n)` *Description*: Turn 'cam0_wrist_x' → 'wrist_x', etc.
- `_simple_beh_labels(names, keypoints)` *Description*: Map raw DLC-style names like: 'DLC_Resnet50_..._wrist_x' -> 'Wrist X' 'cam0_elbow_y' -> 'Elbow Y' If a keypoint/axis can't be found, fall back to the original name.

- `_compute_mean_and_sd(segs)` *Description*: segs : (n_trials, K, T) Returns

  mean : (K, T) sd : (K, T)
- `_get_subset_indices(labels, mode)` *Description*: For "MWT" mode, keep only Middle, Wrist keypoints. labels : list of pretty labels ("Wrist X", "Middle Finger Y", ...)
- `main()`

plot_nobehv.py

Path: batch_scripts/plot_nobehv.py

Description: Generates peri-stimulus population firing rate heatmaps for neural arrays in sessions where behavioral tracking (kinematics) is unavailable or skipped.

Functions:

- `_region_grid(region_name)`
- `render_spatial_grid(ax, data_slice, grid_elec, elec_to_idx, vmin, vmax, smoothing)`
- `_safe_get_scalar_str(x)`
- `main()`

plot_peri_stim_raster.py

Path: batch_scripts/plot_peri_stim_raster.py

Description: Generates single-unit/channel peri-stimulus spike rasters and PSTH line plots around stimulation events to inspect individual channel modulation.

Functions:

- `load_electrode_mapping(csv_path)` *Description*: Load electrode mapping from CSV. Returns: nsp_to_elec: {nsp_id: electrode_id} mapping. region_grids: {region: 8x8_array} of electrode IDs. elec_to_region: {electrode_id: region} mapping.
- `build_elec_to_data_idx(ua_ids_1based, nsp_to_elec)` *Description*: Build electrode_id -> channel index mapping. ua_ids_1based contains NSP IDs (1-256), we need to convert to electrode IDs.
- `_get_raster_data(peak_times_ms, events_ms, win_ms, blank_pre_ms, blank_post_ms)` *Description*: Gather spike times relative to each event. If blank_pre_ms > 0 or blank_post_ms > 0, exclude spikes in [-blank_pre_ms, blank_post_ms] relative to stim event. Returns: trial_indices: list of trial indices for each spike rel_times: list of relative spike times for each spike
- `_rebin_from_peaks(peaks_dict, events_ms, win_ms, bin_ms, stim_dur_ms, blank_pre_ms, blank_post_ms)` *Description*: Re-bin raw peak times into custom-sized bins around events. Returns counts (n_trials, n_channels, n_bins), bin_centers, bin_edges. Bins inside the blanking region [-blank_pre_ms, stim_dur_ms + blank_post_ms] are set to NaN.
- `plot_channel_group(ax_raster, ax_psth, peak_times, events_ms, binned_counts, bin_edges, title, stim_dur_ms, blank_pre_ms, blank_post_ms, bin_centers, n_trials_ref, win_ms, psth_ylim)` *Description*: Plot one channel's raster and PSTH.
- `load_reference_data(all_files, metadata_df)` *Description*: Scans all files for 'control_reaches' and 'at_rest'. Returns: { 'controls': { 'Target_A': {data}, 'Target_B': {data} }, 'rest': { (freq, dur): {data} } }
- `process_file(npz_path, references, metadata_df)`
- `main()`

quantify_stim_kinematics.py

Path: batch_scripts/quantify_stim_kinematics.py

Description: Computes statistical metrics (e.g. latency to movement, movement duration, peak speed, trajectory deviation) of behavioral reaches following stimulation events.

Functions:

- `get_br_filter_for_session()` *Description*: Return list of BR indices to include for current session, or None for all.
- `get_condition_label_from_metadata(br_idx, is_control)` *Description*: Get condition label from metadata CSV. Returns labels like:
  - "Control" for control/baseline conditions (grouped)
  - "40p @ 400Hz (1-32)" for stim conditions Parameters:

  ---

  br_idx : int BR file index is_control : bool Whether this is a control/baseline condition
- `get_br_from_condition_string(cond_str)` *Description*: Extract BR index from condition string like 'Cond_6' or 'BR_006'.

- `is_control_condition(cond_str, br_idx)` *Description*: Determine if a condition is a control/baseline condition. Checks:
    1. Condition string contains 'baseline' or 'control'
    2. Metadata Notes column contains 'baseline' or 'rest'
- `_get_keypoint_indices(all_kp_names, selected_substrings, verbose)` *Description*: Return indices of keypoints that match any substrings. Uses case-insensitive matching for better compatibility across data formats. Also tries matching without underscores for flexibility.
- `_get_keypoint_names_from_data(data, camera)` *Description*: Extract keypoint names from data file, trying multiple possible key formats. Returns list of keypoint names, or empty list if not found.
- `calculate_reach_metrics(pos_segs, vel_segs, ts_state_segs, t_axis, kp_names, target_code, exclude_trials, fs, return_traces, source_file)` *Description*: Calculate duration, peak speed, path length for each trial. Metric: Start at START_OFFSET_MS, End at Max Position Excursion.
- `process_single_file(p, target, target_code)`
- `get_kinematics_id(filename)` *Description*: Generate a unique ID for the behavior session to avoid duplicates. Baseline files: Combine across ports/depths for the same session+target.
    - baseline__NRR_RW011_Depth_43.0_port_A_target_A
    - baseline__NRR_RW011_Depth_43.0_port_B_target_A → Both get ID: "baseline__NRR_RW011_target_A" Peristim files: Each BR index is a SEPARATE condition - NO deduplication.
    - peristim__NRR_RW011_251203_141954__BR_006_target_A
    - peristim__NRR_RW011_251203_141954__BR_007_target_A → Different IDs (full filename preserved)
- `process_target(target, target_code)`
- `combine_conditions(df)` *Description*: Combine conditions 12-16 into one group and 17-21 into another. Returns a new dataframe with combined conditions.
- `detect_outliers(df, target, suffix)` *Description*: Detect outliers in duration_ms using IQR method and save to CSV.
- `plot_individual_points(ax, data_list, conditions)` *Description*: Plot individual data points as hollow circles with jitter.
- `plot_significance(ax, valid_pos, valid_data, baseline_data, conditions, valid_indices)` *Description*: Annotate plot with significance stars (T-test vs Baseline).
- `plot_stim_quantification(df, target, suffix, label_map)`
- `run_target_analysis(target_char, target_code, display_name)` *Description*: Run the full analysis pipeline for a single target:
    - Process kinematics
    - Detect/Remove outliers (optional)
    - Plot individual conditions (optional)
    - Plot combined conditions (optional)
- `main()`

---

2. Core Analysis Functions (`RCP_analysis/python/functions/`)**

These are core, reusable helper modules for data loading, preprocessing, alignment, and utils.

`DLC_BR_alignment.py`

Path: RCP_analysis/python/functions/DLC_BR_alignment.py

Description: Core functions to align DeepLabCut camera frames with Blackrock recordings using frame sync pulse rising edges detected in auxiliary NS5 analog channels.

Functions:

- `simple_label(colname)`
- `detect_rising_edges(sig)`

DLC_BR_alignment_2_cameras_frame_correction.py

Path: RCP_analysis/python/functions/DLC_BR_alignment_2_cameras_frame_correction.py

Description: Extended alignment functions for dual-camera DLC tracking, featuring frame interpolation and dropout correction to handle camera frame drops.

Functions:

- `simple_label(colname)`
- `detect_rising_edges(sig)`

artifact_correction.py

Path: RCP_analysis/python/functions/artifact_correction.py

Description: Implements an Incremental Principal Component Analysis (IPCA) class to learn multi-channel stimulation artifact templates and subtract them dynamically without needing full dataset residency in memory.

Classes:

- `class Template` *Description*: Store IPCA weights for each channel *Methods*:
  - `__init__(self, weights)`
  - `__getitem__(self, index)`
  - `__setitem__(self, index, value)`
  - `__len__(self)`
  - `__repr__(self)`
  - `update_weights(self, new_weights, learning_rate)`
- `class IPCA_Artifact_Correction` *Description*: Artifact correction using Incremental PCA (from sklearn.decomposition) *Methods*:
  - `__init__(self, rank)`
  - `ipca_template_per_channel(self, signal, template, learning_rate)` *Description*: Incremental PCA to correct one channel Input: signal: n_stim * n_time array template: rank * n_time array Returns: signal_corrected: artifact-corrected channel template: updated template with new weights
  - `ipca_all(self, signal)` *Description*: Incremental PCA to correct all channels. Input: signal: n_stim * n_time * n_channels array Returns: signal_all: artifact-corrected signals (n_stim * n_time * n_channels) templates_all: list of Template objects for each channel

    - `apply_template(self, signal, template)` *Description*: Apply existing template to new signal WITHOUT updating the weight! Input: signal: n_stim * n_time array (raw, un-centered) template: Template object with existing weights Returns: signal_corrected: artifact-corrected signal (with baseline preserved)

`artifact_correction_template_matching.py`

Path: RCP_analysis/python/functions/artifact_correction_template_matching.py

Description: Removes stimulation artifacts by identifying artifact shapes via template matching and subtracting matching template waveforms from the raw signal.

Functions:

- `_medfilt_col(args)`
- `_filtfilt_subtract_col(args)`
- `_global_drift_remove_pool(traces, fs, p, n_jobs)` *Description*: Multiprocessing version of global drift removal using Pool. Parallelizes per-channel rolling median and Gaussian smoothing subtraction.
- `_block_window_lengths(trigger_pairs_window, pre, post_pad)` *Description*: For a block: per-pulse target window length = (end + post_pad) - (start - pre) + 1
- `_extract_pulse_snippets(clean, trigger_pairs_window, pre, post_pad, target_len)` *Description*: Extract snippets aligned to pulse start with window [start-pre : end+post_pad] (inclusive). Pads/truncates each to a uniform length L (default: max per-pulse length in for all pulses). Returns

  snips : (n_keep, L, n_channels) float32 keep_mask : (m,) bool L : int
- `_pca_template_per_channel(snips, center)` *Description*: Compute per-channel PCA templates and return a compact PCA pack. Parameters

  snips : (n_pulses, L, n_channels) center : bool If True, subtract timepoint-wise mean across pulses before PCA. Returns

  templ : (L, n_channels) float32 Template reconstructed from top-k PCs for each channel (k = min(3, n_pulses, L)). pca_pack : list of dict Length n_channels; each dict has:
    - 'base': (L,) float32 (p.mean_ if not centered, else timepoint-wise mean)
    - 'components': (k, L) float32 (same orientation as sklearn's components_)
- `_apply_interp_ramp(buf, start_idx, end_idx, frac)` *Description*: Linearly ramp from current edge value to 0 over [start_idx, end_idx).
- `remove_stim_pca_offline(recording, stim_npz_path, params, segment_index)` *Description*: Returns cleaned traces for one segment as a NumPy array (n_samples, n_channels). Steps:
  1. Global drift removal
  2. Pulse-aligned matrices with [-13, +15 after pulse end] windows
  3. PCA template per block (exclude_first_for_pca if requested)

4. Template subtraction (optional per-pulse scaling)
5. Interpulse linear drift correction (artifact end + tail -> next trigger)

- `cleaned_numpy_to_recording(cleaned, recording_like)`

Classes:

- `class PCAArtifactParams`

br_preproc.py

Path: RCP_analysis/python/functions/br_preproc.py

Description: Preprocessing utilities for Blackrock recordings, including reading NSx/NEV files, impedance masking, high-pass filtering, and loading channel mapping.

Functions:

- `list_br_sessions(br_root)` *Description*: List Blackrock sessions under br_root. A session can be either:
  - a subdirectory, or
  - a base filename for NSx/NEV pairs in br_root.
- `ua_excel_path(repo_root, probes_cfg)`
- `extract_br_aux_streams_npz(sess, aux_dir, camera_sync_ch, triangle_sync_ch, touchscreen_ch, hr_ch, vog_sig_ch)` *Description*: Build and save BR aux streams in a unified format into ONE NPZ file:
  - ns5: selected sync channels (camera, triangle, optional extra) stacked as rows.
  - ns2: all channels stacked as rows, plus name-based extraction of: hhpos -> vog_sig vog_sync -> vog_sync touchscreen_syn -> touchscreen_sig heart_rate -> hr_sig Returns

---

(out_npz, touchscreen_sig, hr_sig, vog_sig, meta_ns5, meta_ns2) meta_ns5: dict with ns5 fields (or {} if ns5 missing) meta_ns2: dict with ns2 fields (or {} if ns2 missing) Notes

meta_ns5/meta_ns2 are intended to be merged into a larger meta dict by the caller.

- `apply_ua_mapping_with_regions(recording, mapped_nsp, br_idx, meta_csv, port)` *Description*: Apply UA mapping by renaming channels and build UA-related per-row arrays. Assumptions:
  - metadata CSV has columns: BR_File, UA_port
  - UA_port is 'A' or 'B'
  - recording.get_channel_ids() -> local NSP ids 1..128 (per port)
  - mapped_nsp uses 1..128 for Port A, 129..256 for Port B Parameters

---

port : str or None If 'A' or 'B', overrides the UA_port from the metadata CSV. If None, UA_port is read from meta_csv (BR_File row). Returns

renamed : recording with renamed channel_ids idx_rows : np.ndarray (n_electrodes,), electrode -> recording row index (or -1) ua_elec : np.ndarray (n_channels,), row -> UA electrode number (or -1)

ua_nsp : np.ndarray (n_channels,), row -> NSP id (or -1) ua_region : np.ndarray (n_channels,), row -> region index {-1,0,1,2,3} ua_region_names : np.ndarray (4,), ["SMA", "Dorsal premotor", "M1 inferior", "M1 superior"] ua_port:

- `load_UA_mapping_from_excel(xls_path, sheet, n_elec)` *Description*: Use pandas only to load the sheet, then convert columns to numpy and do all processing in pure numpy.
- `load_electrode_mapping(csv_path)` *Description*: Load electrode mapping from CSV. Returns: nsp_to_elec: {nsp_id: electrode_id} mapping. region_grids: {region: 8x8_array} of electrode IDs. elec_to_region: {electrode_id: region} mapping.
- `get_region_from_group_name(grp_name)` *Description*: Extracts region name from group string like 'M1s (n=45)' -> 'M1s'.
- `get_region_grid(region_name, utah_elec_grids)` *Description*: Gets the 8x8 grid for a given region.
- `build_elec_to_data_idx(ua_ids_1based, nsp_to_elec, ua_port)` *Description*: Builds electrode_id -> channel index mapping. For Port A: data channels 0-127 correspond to NSP 1-128 For Port B: data channels 0-127 correspond to NSP 129-256

`config_loading.py`

Path: RCP_analysis/python/functions/config_loading.py

Description: Loads the global `params.yaml` configuration, resolves absolute directory paths (`REPO_ROOT`, `SESSION_LOC`, etc.), and exposes them as Python constants for the pipeline.

`impedance_utils.py`

Path: RCP_analysis/python/functions/impedance_utils.py

Description: Utilities to parse impedance test logs (e.g. from Utah arrays or Intan) to exclude high-impedance (bad) electrodes from downstream analyses.

Functions:

- `get_bad_channels_utah(file_path, high_threshold, exclude_low_imp)` *Description*: Parses a Utah Array impedance file and returns a set of excluded channel IDs. Args: file_path (Path): Path to the impedance file. high_threshold (float): Impedance value (kOhm) above which to exclude. Default 800. exclude_low_imp (bool): If True, exclude channels <= 15 kOhm. Default True. Criteria:
  - Impedance > high_threshold kOhm
  - Impedance <= 15 kOhm (if exclude_low_imp is True) Format example: elec2-124 15 kOhm elec4-245 <= 15kOhm
- `get_bad_channels_intan(file_path, high_threshold)` *Description*: Parses an Intan impedance CSV and returns a set of excluded channel IDs. Args: file_path (Path): Path to the impedance CSV. high_threshold (float): Impedance value (Ohms) above which to exclude. Default 4.5e6. Criteria:
  - Impedance > high_threshold Ohm
  - ID derived from last 3 digits of 'Channel Name' (col 1, index 1)
- `get_session_impedances(session_loc, utah_high_thresh, utah_exclude_low, intan_high_thresh)` *Description*: Looks for 'Impedances' folder in session_loc and parses standard files. Args: session_loc (Path): Root directory of the session. utah_high_thresh (float): Max impedance for Utah (kOhm). utah_exclude_low (bool): Whether to exclude Utah channels <= 15 kOhm.

intan_high_thresh (float): Max impedance for Intan (Ohm). Returns: dict: {'utah': set(bad_ids), 'nprw': set(bad_ids)}

`intan_preproc.py`

Path: RCP_analysis/python/functions/intan_preproc.py

Description: Functions to load and preprocess Intan raw data files, extract auxiliary analog channels (sync/stim triggers), and perform artifact blanking.

Functions:

- `reorder_recording_to_geometry(rec, perm)` *Description*: Reorder channels of `rec` according to a permutation `perm` which maps from device order to geometric order. Parameters

  rec : BaseRecording Input recording. perm : array-like or None Permutation indices of length rec.get_num_channels(), or None to leave the recording unchanged. Returns

  BaseRecording Channel-sliced recording with reordered channels (or original if perm is None).
- `extract_stim_triggers_and_blocks(stim_data)` *Description*: Detect stimulation pulses and group them into blocks [beg end] Parameters: stim_data : array, shape (n_channels, n_samples) Raw stim stream, zero means baseline Returns: StimTriggerResult object
- `extract_stim_npz(sess, out_dir, stim_stream_name, chanmap_perm)`
- `extract_intan_aux_streams_npz(sess, out_dir, aux_streams)`

Classes:

- `class StimTriggerResult`

`params_loading.py`

Path: RCP_analysis/python/functions/params_loading.py

Description: Helper class/functions to parse and validate YAML experimental configurations (`params.yaml`).

Functions:

- `load_experiment_params(yaml_path, repo_root)`
- `resolve_probe_geom_path(params, repo_root, session_key)` *Description*: Resolve the geometry/mapping .mat path. Priority:
  1. Session-specific probe → mapping_mat_rel or geom_mat_rel
  2. Global params.geom_mat_rel

Classes:

- `class experimentParams`

`utils.py`

Path: RCP_analysis/python/functions/utils.py

Description: General mathematical and signal processing utilities (e.g., spike-rate calculation, Gaussian smoothing, interpolation, peak detection, baseline normalization).

Functions:

- `_parse_condition_cam(path)` *Description*: Take in names such as: NRR_RW007_001_[date]*Cam-0_ocr.csv NRR_RW007_001*[date]*Cam-1DLC_Resnet50*....csv and outputs the condition ID, camera #, and DLC or OCR file
- `find_per_cond_inputs(video_root)` *Description*: Scan VIDEO_ROOT/DLC for OCR/DLC files and keep newest file. Return conditions that have both cams for both 'ocr' and 'dlc'
- `get_metadata_mapping(meta_csv, field1, field2)` *Description*: field1 -> field2 mapping.
  - keys are int
  - values are stripped strings (no casting)
- `detect_IR_crossings(x, fs, refractory_sec)`
- `load_ocr_map(ocr_csv)` *Description*: Load OCR file
- `load_dlc(dlc_csv)` *Description*: Load DLC file. If the first column is 'frame' (in any header level), use it as the AVI_framenum index; otherwise use 0..N-1. Columns are flattened and cast to numeric where possible. TODO
- `align_dlc_to_corrected(dlc_df, ocr_df)` *Description*: Map DLC rows (indexed by AVI_framenum) into a DataFrame indexed by CORRECTED_framenum. Dropped frames become NaNs. TODO
- `frame2sample_br_ns5_sync(n_corrected, ns_path, sync_chan)` *Description*: Read Blackrock NS5, detect rising edges on, and return sample indices corresponding to the frames. Returns: samples: (n_corrected,) int32
- `frame2sample_br_ns2_sync(n_corrected, ns_path, sync_chan)` *Description*: Read Blackrock NS2, detect rising edges on `sync_chan`, and return sample indices corresponding to the frames. Parameters

  n_corrected : int Number of corrected frames (length of the desired mapping). ns_path : Path Path to the .ns2 file. sync_chan : int Channel id in the NS2 file carrying the VOG sync signal. Returns

  samples : (n_corrected,) int32 Sample indices of the rising edges, one per corrected frame.
- `_is_ns5_col(name)`
- `_flatten_cols_mi(mi)` *Description*: Flatten possibly-messy MultiIndex columns:
  - Drop 'nan' / 'Unnamed.*' levels
  - Join remaining parts with '_'
  - Normalize any ns5-sample-like column to 'ns5_sample'
- `_read_behavior_csv_robust(csv_path, num_cam)` *Description*: If num_cam > 1, read as a 2-level header (for both-cam CSVs). Else read as a single header (for single-cam CSVs). Always normalize any ns5* columns to 'ns5_sample'.
- `load_behavior_npz(csv_path, num_cam)` *Description*: Reads aligned behavior CSV and returns: ns5_sample: (N,) int64 cam0: (N, D0) float32, cam0_cols: list[str] cam1: (N, D1) float32, cam1_cols: list[str] num_cam: 1 or 2. Function is tolerant to files that only include one cam even when num_cam=2.

- `list_intan_sessions(root)`
- `save_recording(rec, out_dir)`
- `load_intan_aux(npz_path)`
- `extract_peristim_segments(rate_hz, counts, t_ms, stim_ms, win_ms, min_trials)` *Description*: Return: rate_segments : (n_trials, n_ch, n_twin) count_segments : (n_trials, n_ch, n_twin) or None if counts is None rel_time_ms : (n_twin,) stim_ms_valid : (n_trials,) subset of stim_ms that produced valid segments Skips triggers whose window falls outside t_ms or whose slice length doesn't match the expected window length.
- `baseline_zero_each_trial(segments, rel_time_ms, normalize_first_ms)` *Description*: For each trial & channel, subtract the mean over the first `normalize_first_ms` of the segment (i.e., from window start to start+normalize_first_ms). segments: (n_trials, n_ch, n_twin)
- `median_across_trials(zeroed_segments)` *Description*: Median across trials per channel×time, ignoring NaNs. If a (ch, t) bin is all-NaN across trials, the result stays NaN. Input: (n_trials, n_ch, n_t) -> (n_ch, n_t)
- `variance_across_trials(zeroed_segments)` *Description*: Variance across trials per channel×time, ignoring NaNs. If a (ch, t) bin is all-NaN across trials, the result stays NaN. Input: (n_trials, n_ch, n_t) -> (n_ch, n_t)
- `load_stim_detection(npz_path)` *Description*: Required fields:
  - trigger_pairs : (n_pulses, 2) [start, end] in Intan index space
  - block_bounds_samples : (n_blocks, 2) [start, end] in Intan index space
  - pulse_sizes : (n_pulses,) Optional:
  - active_channels : (n_active,)
- `stim_npz_path_from_br_idx(br_idx, mapping_csv, nprw_aux_root)` *Description*: Given a BR_File index (br_idx), return the corresponding stim_stream.npz path for the matching Intan session, or None if anything cannot be resolved.
- `detect_stim_channels_from_npz(stim_npz_path, eps, min_edges)` *Description*: Return GEOMETRY-ORDERED indices of stimulated channels by counting 0→nonzero rising edges over the full 'stim_traces' array stored in the NPZ. NaN-safe: ignores channels with no finite data.
- `aligned_stim_ms(stim_ms_abs, meta)` *Description*: Convert absolute Intan stim times (ms) into the aligned timebase used in the combined file: intan_t_ms_aligned = intan_t_ms - t0_intan_ms where t0_intan_ms = shift_sample * 1000 / fs_intan
- `ua_title_from_meta(meta)` *Description*: Build Utah array title from metadata.
- `parse_intan_session_dtkey(session)`
- `build_session_index_map(intan_sessions)`
- `find_ns5_by_br_index(br_root, br_idx)` *Description*: Locate an .ns5 file whose stem ends with the 3-digit BR index, e.g.: br_idx = 1 -> token '001' -> matches '*001.ns5' If multiple matches exist, prefer:
  1. Newest modification time
  2. Largest size
- `find_ns2_by_br_index(br_root, br_idx)` *Description*: Locate an .ns2 file whose *stem* ends with the 3-digit BR index, e.g.: br_idx = 1 -> token '001' -> matches '*001.ns2' If multiple matches exist, prefer:
  1. Newest modification time
  2. Largest size
- `_butter_lowpass_ba(cutoff_hz, fs_hz, order)`
- `_filtfilt_nanaware(x, b, a)` *Description*: Filter 1D array with NaNs:
  - keep a mask of NaN samples
  - fill with column median

    - filtfilt
    - put NaNs back
- `_central_diff_ms(x, dt_ms)` *Description*: Central difference derivative along time for 1D array. Respects NaNs: any side that is NaN → derivative set to NaN there.
- `butter_lowpass_pos_and_vel(TxD, t_ms, cutoff_hz, order)`
- `butter_lowpass_pos_and_vel_3d(NKT, t_ms, cutoff_hz, order)`
- `dedup_peaks(peaks, amps, dedup_ms, max_cluster_ms)` *Description*: Deduplicate peaks per (segment, channel), keeping strongest within short windows
- `bin_counts_around_stim(peaks_ms, bin_ms, stim_times_ms, art_before_ms, art_after_ms, win_ms)` *Description*: Bin spike counts around each stimulation event.
- `_smooth_segment(seg_counts, seg_edges, seg_centers, sigma_ms)`
- `smooth_counts_gauss(counts, edges, bin_centers_ms, sigma_ms, left_bins)` *Description*: Smooth spike counts -> firing rates (Hz) separately on left and right sides of the stim-blank gap, using a Gaussian in ms (NaN-aware at the per-bin level).

---

3. Core Plotting Functions (`RCP_analysis/python/plotting/`)**

These modules contain reusable plotting functions for visualizing results.

`plotting.py`

Path: RCP_analysis/python/plotting/plotting.py

Description: Core plotting routines for the project, including generating stacked heatmaps (neural population PSTHs stacked by brain region/depth) synchronized with kinematic traces and shank/array layout maps.

Functions:

- `add_ua_region_bar(ax, n_rows, ua_chan_ids_1based, x, width)` *Description*: Draw a contiguous vertical bar subdivided into the 4 UA regions. If ua_chan_ids_1based is provided, segment heights are proportional to how many rows belong to each region; otherwise equal quarters.
- `stacked_heatmaps_plus_behv(nprw_med, ua_med, t_nprw, t_ua, nprw_edges_ms, ua_edges_ms, out_svg, title_kinematics, title_NA)` *Description*: s: session ID

---

4. Analysis & Helper Scripts (`scripts/`)**

Root Scripts

General scripts directly located in the `scripts/` directory:

`analyze_and_plot_FR_UA_NPRW.ipynb`

Path: scripts/analyze_and_plot_FR_UA_NPRW.ipynb

Description: Jupyter notebook to interactively compute, compile, and visualize firing rate changes across Utah Array and Neuropixels arrays.

Functions:

- `run_scripts(base_dir)`

`analyze_and_plot_FR_UA_NPRW.py`

Path: scripts/analyze_and_plot_FR_UA_NPRW.py

Description: Script to compute, compile, and visualize firing rate changes across Utah Array and Neuropixels arrays.

Functions:

- `run_scripts(base_dir)`
- `main()`

`analyze_stim_reach.py`

Path: scripts/analyze_stim_reach.py

Description: Analyzes reach kinematics and neural modulations under different stimulation conditions, generating multi-panel peri-stimulus population figures.

Functions:

- `_pick_rates_all_path(rates_dir, ua_port_choice)` *Description*: Prefer, in order:
    1. CURATED, port-specific, session-aggregated (no *_BR*)
    2. ALL, port-specific, session-aggregated (no *_BR*)
    3. last ALL of anything (per-BR, etc.)
- `_filter_stims_for_stream(stim_ms, t_ms, win_ms)` *Description*: Keep only stims whose [stim+win0, stim+win1] lies fully within t_ms range.
- `_safe_extract_segments(rate_hz, t_ms, stim_ms_in, win_ms, min_trials, normalize_first_ms)` *Description*: Filter to in-range stims, then extract without crashing if 0-kept.
- `_safe_extract_segments_stat(rate_hz, t_ms, stim_ms_in, win_ms, min_trials, normalize_first_ms, stat)` *Description*: Like _safe_extract_segments, but returns an across-events statistic ("median" or "var") over baseline-zeroed segments (axis=0). Returns (stat_mat, rel_t, n_trials_kept).
- `_order_rows_by_region_then_peak(ua_mat, t_vec, ids_1based)` *Description*: Sort rows within each region by earliest *time* of peak in the selected window. Tie-breaker: larger peak magnitude (by peak_mode). Rows with no finite values go to the bottom of their region. Unknown-region rows go after known regions, sorted the same way. If earliest_at_top=True, each region's block is reversed so that with imshow(origin="lower") the earliest peaks appear at the top.
- `_unit_to_kohm(val, unit)`
- `_read_text_loose(p)`
- `_ylabel_horizontal(ax, text, pad)`
- `load_impedances_from_textedit_dump(path_like)` *Description*: Parse lines like 'elec1-5 201 kOhm' from a loose text dump. Returns { Elec#: impedance_kΩ }.
- `_apply_relative_offsets_by_ref(lines, labels, keypoints, ref_kp)` *Description*: For position lines (K,T), add a constant offset per trace so that: reference_kp_{x,y} → offset 0, other kp_{x} →

+ (median(kp_x) - median(reference_kp_x)), other kp_{y} → + (median(kp_y) - median(reference_kp_y)). If ref_kp is None, the first keypoint from `keypoints` that appears in labels is used.

- `_gaussian_fir_coeffs(sigma_ms, dt_ms, truncate)` *Description*: Build a discrete Gaussian FIR kernel (normalized) with std = sigma_ms, truncated at +/- truncate*sigma, length odd.
- `_filtfilt_gaussian_nanaware_1d(y, b)` *Description*: NaN-aware zero-phase Gaussian smoothing using filtfilt with FIR 'b' (a=[1]).
- `_smooth_lines_gaussian_filtfilt(lines, rel_t_ms, sigma_ms)` *Description*: Apply filtfilt-based Gaussian smoothing along time to (K,T) or (T,) arrays.
- `_butter_lowpass_ba(cutoff_hz, fs_hz, order)`
- `_filtfilt_nanaware(x, b, a)` *Description*: Filter 1D array with NaNs:
  - keep a mask of NaN samples
  - fill with column median
  - filtfilt
  - put NaNs back
- `_central_diff_ms(x, dt_ms)` *Description*: Central difference derivative along time for 1D array. Respects NaNs: any side that is NaN → derivative set to NaN there.
- `butter_lowpass_pos_and_vel(TxD, t_ms, cutoff_hz, order)`
- `butter_lowpass_pos_and_vel_3d(NKT, t_ms, cutoff_hz, order)`
- `_ua_region_code_from_elec(e)` *Description*: Return 0..3 for the 4 UA regions (SMA, DPM, M1 inf, M1 sup). Unknown -> large code.
- `_simple_beh_labels(names, keypoints)` *Description*: Map raw DLC-style names like 'DLC_Resnet50_..._wrist_x' -> 'wrist_x' 'cam0_elbow_y' -> 'elbow_y' If a keypoint/axis can't be found, fall back to the original name.
- `_baseline_zero_trials(wins, rel_t_ms, normalize_first_ms)` *Description*: wins: (N_events, K_traces, T) — baseline-zero per event/trace using [rel_t_ms[0], normalize_first_ms].
- `_find_baseline_cam_npz(root, port, depth, cam)` *Description*: Prefer curated → non-curated. Accept legacy naming without _camX as last fallback.
- `_load_baseline_cam(npz_path)`
- `main_baselines()` *Description*: Baseline figure styled like stacked_heatmaps_plus_behv (no probe): [Kinematics median lines] + [Intan heatmap] + [UA heatmap]
- `_as_list(x)`
- `_median_behavior_line(series_on_common, t_common_ms, stim_ms, win_ms, baseline_ms, min_trials)`
- `_median_lines_for_columns(series_on_common, t_common_ms, stim_ms)`
- `_ordered_xy_indices(cam_cols, keypoints)`
- `_strip_cam_prefix(n)`
- `_select_matrix(cam, cols, keypoints)`
- `_z_per_column(M)`
- `_parse_delay_ms(val)`
- `_fmt_num(x)`
- `_interp_nans_1d_gentle(x, max_gap)` *Description*: Fill only interior NaN runs up to max_gap by linear interp. Leading/trailing NaNs stay NaN; large gaps stay NaN.
- `_interp_nans_2d_by_col(arr, max_gap)` *Description*: Apply _interp_nans_1d_gentle to each column of a 2D array.
- `_read_csv_robust(path)` *Description*: Try several common encodings and tolerate a few bad rows.
- `build_title_from_csv(csv_path)`

- `load_events(session_name)`
- `filter_stims_by_reach(stim_ms, events, target_type, window_sec)` *Description*: Keep stims where a reach of 'target_type' ('A' or 'B') starts within window_sec after stim.
- `main()`

`analyze_touchscreen_buttons.py`

Path: scripts/analyze_touchscreen_buttons.py

Description: Epochs behavioral and neural data around touchscreen button presses (Buttons A and B) to generate aligned PSTH heatmaps and kinematic plots.

Functions:

- `load_aligned_data(fpath)`
- `load_events(session_name)`
- `extract_epochs(data_continuous, trigger_indices, win_samples)` *Description*: Extract epochs from (N_time, N_ch) data. Returns (N_trials, N_ch, N_time)
- `get_channel_depths_intan()`
- `plot_session(sess_name, aligned, events, fs_events)`
- `main()`

`combine_UA_gifs.py`

Path: scripts/combine_UA_gifs.py

Description: Stitches separate spatial LFP/firing rate heatmaps of the 4 Utah Arrays (SMA, PMd, M1i, M1s) into a single anatomically arranged quad GIF showing spatial dynamics.

Functions:

- `combine_gifs(session_id, datatype, mode, region_paths)` *Description*: Combines 4 array GIFs into an anatomical quad view with consistent scaling.
- `main()`

`debug_IPCA_artifact_correction.ipynb`

Path: scripts/debug_IPCA_artifact_correction.ipynb

Description: Jupyter notebook to interactively evaluate IPCA artifact subtraction parameters and visualize raw vs. cleaned waveforms.

Functions:

- `find_aligned_file(aligned_dir, br_idx)`
- `find_intan_folder(intan_root, intan_filename)`
- `get_probe_geometry()`
- `run_ipca_debug(condition, probe, target_ch, ipca_rank, window_ms, trials_plot, pulse_window_ms, apply_hpf, debug_single_pulse, stim_freq_override,`

`use_behavior_filter, ua_refine_search_ms)`

`debug_IPCA_artifact_correction.py`

Path: scripts/debug_IPCA_artifact_correction.py

Description: Interactive/debug script to evaluate IPCA artifact subtraction parameters and visualize raw vs. cleaned waveforms.

Functions:

- `find_aligned_file(aligned_dir, br_idx)` *Description*: Locate an aligned `.npz` across standard checkpoint subfolders.
- `find_intan_folder(intan_root, intan_filename)` *Description*: Resolve the Intan recording folder by exact or partial name match.
- `load_probe_geometry()` *Description*: Load the Neuropixels channel map from the session's .mat file.
- `_find_first_significant_peak(signal_chunk)` *Description*: Return the index of the *earliest* significant deflection in a chunk. The chunk is mean-subtracted so that baseline drift does not influence the result. A 40 % threshold relative to the local maximum is used to reject noise, and the peak of the first supra-threshold region is returned. This function is critical for consistent alignment of biphasic artifacts: it always locks onto the same (first) phase regardless of drift.
- `_exp_model(t, A, tau)` *Description*: Single-term exponential decay model: A * exp(-t / tau).
- `run_ipca_debug(condition, probe, target_ch, ipca_rank, window_ms, trials_to_plot, pulse_window_ms, apply_hpf, debug_single_pulse, use_behavior_filter, ua_refine_search_ms, reference_ch, exp_subtract, exp_fit_guard_ms, exp_n_tau_fits, exp_tau_bounds_ms, exp_tau_seed_ms, plot_rank_variance, plot_learning_rate_sweep, override_freq, per_channel_ipca, nprw_block_size, exclude_nprw_stim_channels)` *Description*: Run the full IPCA artifact-correction debug pipeline. Parameters

  condition : int Blackrock block index to analyse. probe : str "NPRW" (Neuropixels) or "UA" (Utah Array). target_ch : int or str Channel number to analyse (1-based electrode ID for UA), or "all" to process every mapped channel with cross-channel IPCA. ipca_rank : int Number of principal components used to model the artifact. window_ms : tuple of float (pre, post) extraction window around each stim block (ms). trials_plot : int Maximum number of trials to include in the summary figure. pulse_window_ms : tuple of float (pre, post) micro-window around each detected pulse (ms). apply_hpf : bool If True, apply a 300 Hz high-pass filter before extraction. debug_single_pulse : bool If True, save a diagnostic overlay of all micro-windows. use_behavior_filter : bool If True, restrict to behaviourally valid Target A trials. ua_refine_search_ms : float Search

radius (ms) for pulse centre refinement on the UA trace. reference_ch : int or None When target_ch="all", which channel to use for pulse detection. Defaults to the first mapped channel. plot_rank_variance : bool If True and running in single-channel mode, plots the Cumulative Explained Variance vs IPCA Rank to help find the optimal elbow. plot_learning_rate_sweep : bool If True and running in single-channel mode, sweeps learning rates (0.1 to 1.0) and plots the residual temporal variance/MSE. Returns

dict All extracted arrays and metadata for downstream analysis.

Nikita's Scripts

Scripts created by Nikita (`scripts/nikita_scripts/`):

`debug_lfp_pipeline_plots.py`

Path: scripts/nikita_scripts/lfp_processing/debug_lfp_pipeline_plots.py

Description: Verification script to plot LFP spectral densities and filtering steps to inspect pipeline efficacy.

Functions:

- `plot_debug_step_colored(rec, step_name, stim_indices, pre_ms, post_ms, save_dir)` *Description*: Plots a snippet of data around the first stimulus event with channel coloring. Channels 1-16 -> Color 1, 17-32 -> Color 2, etc.
- `indent_colors()`
- `run_debug_pipeline(sess_name, stim_indices_override, shift_ms)`
- `plot_epoch_snapshot(traces, step_name, fs, pre_ms, x_lim, save_dir, title_suffix)` *Description*: Plot helper for epoch data (n_ch, n_time)
- `run_baseline_debug_pipeline(npz_path)` *Description*: Process baseline files by loading IR events from PeriStim baseline files and using them as stim_indices for the debug pipeline. Approach mirrors analyze_lfp_bands.py lines 730-766:
    1. Extract contributing sessions from aligned baseline LFP file
    2. Load IR events from corresponding PeriStim baseline file
    3. Convert IR times to native samples
    4. Run debug pipeline on representative session with those IR events

Classes:

- `class BlankingRecording` *Methods*:
    - `__init__(self, parent_recording, stim_indices, pre_ms, post_ms, mode)`
- `class BlankingRecordingSegment` *Description*: Custom SI segment that zeros out or interpolates over stir-related artifacts defined by stim_indices. *Methods*:
    - `__init__(self, parent_segment, stim_indices, pre_samples, post_samples, mode)`
    - `get_num_samples(self)`
    - `get_traces(self, start_frame, end_frame, channel_indices)`

- `class TimeReversedRecordingSegment` *Methods*:
  - `__init__(self, parent_segment)`
  - `get_num_samples(self)`
  - `get_traces(self, start_frame, end_frame, channel_indices)`
- `class TimeReversedRecording` *Methods*:
  - `__init__(self, parent_recording)`

`plot_lfp_all.py`

Path: scripts/nikita_scripts/lfp_processing/plot_lfp_all.py

Description: Orchestrates loading preprocessed LFP data, computing trial-averaged spectrograms, and plotting spatial LFP response heatmaps.

Functions:

- `_region_grid(region_name)`
- `get_representative_channel(data_array)` *Description*: Find a channel index that has valid, non-flat data. data_array: (n_trials, n_ch, n_time)
- `calculate_psd(data, fs)` *Description*: Calculate Power Spectral Density averaged across trials. Returns: freq, psd_mean (n_ch, n_freq), psd_sem (n_ch, n_freq) data: (n_trials, n_ch, n_time)
- `calculate_coherence(data, rep_ch_idx, fs)` *Description*: Calculate Magnitude Squared Coherence between rep_ch and all other channels. Returns: freq, coh_mean (n_ch, n_freq) - averaged across trials
- `calculate_itpc(data, fs)` *Description*: Inter-Trial Phase Coherence. Returns: freq, itpc (n_ch, n_freq) Using Morlet Wavelets or Hilbert? Given the short windows, let's use FFT-based ITPC (Phase consistency of Fourier components).
- `calculate_tf_itpc(data, fs, nperseg, noverlap)` *Description*: Calculate Time-Frequency Inter-Trial Phase Coherence using Short-Time FFT. Returns: f, t, itpc_matrix (n_ch, n_freq, n_time_bins) data: (n_trials, n_ch, n_time)
- `calculate_ersp(data_pre, data_post, fs, nperseg, noverlap)` *Description*: Calculate Event-Related Spectral Perturbation (ERSP) in dB. Returns: f, t_post, ersp_matrix (n_ch, n_freq, n_time_bins) normalized to baseline (data_pre).
- `compute_ersp_spectrogram(broadband_full, t_full_ms, fs, spec_cfg)` *Description*: Compute per-channel ERSP spectrogram from broadband_full data. Args: broadband_full: (n_trials, n_ch, n_time) broadband LFP epochs t_full_ms: (n_time,) time axis in ms fs: sampling rate (Hz) spec_cfg: dict — see SPECTROGRAM SETTINGS REFERENCE above Returns: If return_power_trials is False: ersp: (n_ch, n_freq_crop, n_time_crop) power in dB freqs: (n_freq_crop,) frequency axis t_bins_ms: (n_time_crop,) time axis in ms If return_power_trials is True: ersp, freqs, t_bins_ms, P (per-trial power) NOTE: When return_power_trials=True, time is NOT cropped to time_range so that baseline normalization can be applied later.
- `extract_band_power_trials(P, f, band_name, baseline_mask)` *Description*: Extracts per-trial band power over time and converts to dB relative to baseline. P: (n_trials, n_ch, n_freq, n_time) f: (n_freq,) band_name: str baseline_mask: (n_time,) bool
- `plot_band_overlays(band_dB, fs, t_ms, ua_ids_1based, session, fig_dir, config, groups, nsp_to_elec, ua_port)`

- `render_spatial_grid(ax, mean_band_dB, t_bin_idx, grid_elec, elec_to_idx, vmin, vmax, smoothing)` *Description*: Renders a spatial grid onto an axis, optionally with smoothing (interpolation).
- `plot_spatial_heatmaps(band_dB, t_ms, ua_ids_1based, session, fig_dir, config, groups, nsp_to_elec, ua_port)`
- `generate_spatial_gif(band_dB, t_ms, ua_ids_1based, session, fig_dir, config, groups, nsp_to_elec, ua_port)` *Description*: Generate an animated GIF of the spatial heatmap over time.
- `calculate_band_power_stats(data_pre, data_post, fs)` *Description*: Compare band power between Pre and Post using Paired T-Test. Returns: Dict of results per band
- `add_band_annotations(ax)` *Description*: Adds vertical spans for standard LFP bands.
- `calculate_csd(lfp_mean, y_coords)` *Description*: Computes 1D Current Source Density (CSD) from LFP using interpolation to handle irregular spacing/gaps. Parameters: lfp_mean: (n_ch, n_time) - Mean LFP voltage y_coords: (n_ch,) - Depth coordinate for each channel Returns: csd: (n_grid-2, n_time) - CSD values on uniform grid csd_depths: (n_grid-2,) - Depth coordinates for CSD rows
- `process_directory(lfp_dir, fig_dir, label)`
- `plot_lfp_check()`

`learning_artifact_waveform.py`

Path: scripts/nikita_scripts/plotting_codes/learning_artifact_waveform.py

Description: Extracts stimulation artifact waveforms from baseline recordings to train matched-filter detection templates.

Functions:

- `find_aligned_file(aligned_dir, br_idx)` *Description*: Locate an aligned .npz across standard checkpoint subfolders.
- `find_intan_folder(intan_root, intan_filename)` *Description*: Resolve the Intan recording folder by exact or partial name match.
- `load_probe_geometry(geom_path)` *Description*: Load the Neuropixels channel map from the session's .mat file.
- `_find_first_significant_peak(signal_chunk)` *Description*: Return the index of the earliest significant deflection in a chunk.
- `main()`

`plot_blackrock_contents.ipynb`

Path: scripts/nikita_scripts/plotting_codes/plot_blackrock_contents.ipynb

Description: Notebook for visualizing Blackrock file data channels, spectra, and metadata.

`plot_impedances_over_time.py`

Path: scripts/nikita_scripts/plotting_codes/plot_impedances_over_time.py

Description: Parses historical impedance logs to track and plot Utah Array electrode impedance health over multiple days/sessions.

Functions:

- `parse_intan(file_path)`
- `parse_utah(file_path)`
- `plot_utah_impedances(data_list, name, vmax, out_path)` *Description*: Plot Utah array impedances over time with regional breakdown
- `plot_region_trends(data_list, name, out_path)` *Description*: Line plot showing median impedance and % good channels per region over time
- `main()`

`plot_individual_kinematics.py`

Path: scripts/nikita_scripts/plotting_codes/plot_individual_kinematics.py

Description: Plots individual raw and smoothed kinematic trajectories (position and velocity) for manual quality control of reach trials.

Functions:

- `_strip_cam_prefix(n)`
- `_simple_beh_labels(names, keypoints)`
- `filter_traces(segs, names)` *Description*: segs: (n_features, n_time) names: list of feature names Returns filtered_segs, filtered_names
- `plot_single_trial(ax, t, segs, names, title_prefix)` *Description*: segs: (n_features, n_time) names: list of feature names
- `find_all_peristim_files()` *Description*: Find all PeriStim .npz files in the updated directory structure. Returns: List of tuples: (file_path, condition_type, target_label)
  - condition_type: 'stim', 'control', or 'continuous_stim'
  - target_label: 'target_A', 'target_B', or 'continuous_stim'
- `process_single_file(peri_stim_npz_loc, condition_type, target_label)` *Description*: Process a single PeriStim file and generate the trial visualization.
- `main()`

`plot_ua_hpf_traces.py`

Path: scripts/nikita_scripts/plotting_codes/plot_ua_hpf_traces.py

Description: Visualizes raw high-pass filtered voltage traces around stimulation events to inspect artifact subtraction performance.

Functions:

- `find_aligned_file(aligned_dir, br_idx)` *Description*: Find the aligned .npz for a given BR index across all subdirectories.
- `find_pp_folder_for_intan(ckpt_dir, intan_filename, probe, condition)` *Description*: Find the preprocessed SpikeInterface folder that matches a specific Intan session filename or Blackrock condition index.

- `extract_traces(rec, ch_row, center_ms, win_ms, fs)` *Description*: Extract one trace window (in µV) for a channel around a stim event.
- `main()`

`quantify_endpoint_error.py`

Path: scripts/nikita_scripts/plotting_codes/quantify_endpoint_error.py

Description: Calculates endpoint accuracy, path straightness, peak speed, and trajectory metrics from 3D kinematics.

Functions:

- `load_2d_dlc_likelihoods(dlc_2d_root, condition_name, keypoint)` *Description*: Load likelihood values for a keypoint from both camera views. Args: dlc_2d_root: Path to Video/DLC/ directory condition_name: e.g., "NRR_RW022_007" keypoint: Body part name (e.g., "middle", "TA", "TB") Returns: likelihood_cam0: (N,) array of likelihoods from camera 0 likelihood_cam1: (N,) array of likelihoods from camera 1
- `get_likelihood_mask(dlc_2d_root, condition_name, keypoint, frame_indices, threshold)` *Description*: Create a mask where True = VALID (both cameras have likelihood >= threshold). Args: dlc_2d_root: Path to Video/DLC/ directory condition_name: Condition name (e.g., "NRR_RW022_007") keypoint: Body part name frame_indices: Frame indices to check threshold: Minimum likelihood (default 0.4) Returns: valid_mask: (N,) boolean array, True where data is reliable n_cam0_low: Number of frames with low cam0 likelihood n_cam1_low: Number of frames with low cam1 likelihood
- `apply_likelihood_mask(xyz, valid_mask)` *Description*: Set xyz values to NaN where likelihood is below threshold. Args: xyz: (N, 3) position array valid_mask: (N,) boolean array, True = valid Returns: xyz_masked: (N, 3) array with invalid points set to NaN
- `remove_high_velocity_points(xyz, fps, max_speed)` *Description*: Simple and direct: compute velocity, remove any frame where velocity > max_speed. Args: xyz: (N, 3) position array fps: Frames per second max_speed: Maximum allowed speed (units per second) Returns: xyz_clean: (N, 3) with bad frames set to NaN outlier_mask: (N,) boolean, True = removed
- `remove_bracketed_outlier_regions(xyz, fps, max_speed, max_region_frames)` *Description*: Remove entire regions that are bracketed by high-velocity jumps. If we detect: normal -> [high velocity jump] -> garbage -> [high velocity jump] -> normal Then the entire garbage region (including the boundary points) gets removed. Args: xyz: (N, 3) position array fps: Frames per second max_speed: Velocity threshold for detecting jumps max_region_frames: Maximum size of region to remove (safety limit) Returns: xyz_clean: (N, 3) with bad regions set to NaN outlier_mask: (N,) boolean, True = removed
- `parse_args()` *Description*: Parse command line arguments.
- `get_process_only_list(args)` *Description*: Determine which BR indices to process.
- `load_dlc_3d(dlc_3d_path)` *Description*: Load 3D DLC CSV with multi-level header.
- `get_xyz(df, keypoint, frame_idx)` *Description*: Extract x, y, z coordinates for a keypoint at given frame indices.
- `interpolate_small_gaps(arr, max_gap)` *Description*: Linearly interpolate NaN gaps up to max_gap consecutive frames.
- `clean_finger_signal_v2(xyz, fps, rel_time_ms, max_speed, max_interp_gap, position_outlier_std)` *Description*: Clean finger position signal by:

    1. Finding high-velocity jumps
    2. Marking entire regions between jump-pairs as bad
    3. Removing position outliers
    4. Only interpolating small gaps (not cleaned regions)
- `create_oscillation_summary_plot(df, output_dir, summary_br_list)` *Description*: Plot oscillation metrics by condition.
- `compute_direction_changes_v2(xyz, rel_time_ms, start_time_ms, end_time_ms, min_reversal_magnitude, min_reversal_duration_ms, smoothing_window)` *Description*: Detect direction changes by finding local maxima/minima (peaks/valleys) in the primary movement axis position signal. A direction change is a peak or valley where:
    - The position change from previous extremum exceeds min_reversal_magnitude
    - Sufficient time has passed since last reversal Args: xyz: (N, 3) position array rel_time_ms: (N,) time array relative to event start_time_ms: Start of analysis window end_time_ms: End of analysis window (None = use all data) min_reversal_magnitude: Minimum position change to count as reversal min_reversal_duration_ms: Minimum time between consecutive reversals smoothing_window: Light smoothing window (frames) - set to 1 for no smoothing Returns: dict with direction change metrics
- `find_return_to_start(xyz, rel_time_ms, start_time_ms, return_threshold_pct, min_search_time_ms)` *Description*: Find when the hand returns close to its starting position. This extends the analysis window beyond just the endpoint to capture the full reach-and-return movement. Args: xyz: (N, 3) position array rel_time_ms: (N,) time array start_time_ms: Time to use as "start" reference return_threshold_pct: How close to start counts as "returned" (% of max excursion) min_search_time_ms: Don't look for return before this time Returns: return_idx: Index of return point return_time_ms: Time of return status: Description of result
- `smooth_signal(arr, window)` *Description*: Smooth a 1D signal using uniform (moving average) filter. Uses scipy.ndimage which is more robust than savgol_filter for edge cases. Args: arr: 1D array to smooth window: Smoothing window size (must be odd for symmetry) Returns: Smoothed array
- `compute_velocity(xyz, dt_sec, smooth, smooth_window)` *Description*: Compute instantaneous velocity magnitude from 3D positions. Args: xyz: (N, 3) position array dt_sec: Time step in seconds smooth: Whether to apply smoothing smooth_window: Smoothing window size (default: 3 for light smoothing) Returns: velocity: (N,) array of velocity magnitudes
- `euclidean_distance(a, b)` *Description*: Compute Euclidean distance between two sets of 3D points.
- `get_target_position_for_reach(target_xyz, method)` *Description*: Compute a representative target position for a reach window.
- `compute_path_length(xyz)` *Description*: Compute total path length (sum of frame-to-frame distances).
- `compute_trajectory_straightness(xyz)` *Description*: Compute trajectory straightness (direct distance / path length). 1.0 = perfectly straight, < 1.0 = curved
- `compute_velocity_dip(velocity, rel_time_ms, search_start_ms, search_end_ms)` *Description*: Detect velocity dip (deceleration) during reach - characteristic of stim effect.
- `compute_jerk_metric(velocity, dt_sec)` *Description*: Compute mean absolute jerk (derivative of acceleration) - measure of movement smoothness.
- `check_reach_direction_v2(xyz, rel_time_ms, check_window_ms, approach_required, target_pos)` *Description*: Check if the hand is moving TOWARD the target using the primary movement axis. Returns: is_approaching: True if moving toward target net_change: Net distance change

(negative = approaching) reason: Description primary_axis: Which axis (0=x, 1=y, 2=z) had most movement

- `find_first_closest_approach(distance, rel_time_ms, min_approach_time_ms, first_approach_threshold)` *Description*: Find the first closest approach AFTER the minimum approach time.
- `safe_nanmax(arr)` *Description*: Safe nanmax that returns nan for all-nan arrays.
- `safe_nanmean(arr)` *Description*: Safe nanmean that returns nan for all-nan arrays.
- `process_single_reach(dlc_df, event_frame, win_frames, fps, finger_keypoint, target_keypoint, dlc_2d_root, condition_name, debug)` *Description*: Process a single reach to compute endpoint error and trajectory metrics.
- `find_condition_name_for_br(br_idx)` *Description*: Find the condition name for a given BR index.
- `process_peristim_file(peristim_path, dlc_3d_root, dlc_2d_root, behv_aligned_root, condition_type, target_filter, debug)` *Description*: Process a single PeriStim file to compute endpoint errors for all reaches.
- `print_session_summary(combined_df)` *Description*: Print a detailed summary of reach processing results.
- `plot_single_reach_diagnostic(dlc_df, event_frame, win_frames, fps, reach_idx, condition_name, output_dir, reach_result, finger_keypoint, target_keypoint, dlc_2d_root)` *Description*: Create diagnostic plot for a single reach with position-based direction change markers.
- `create_approach_and_error_summary(df, output_dir, summary_br_list)` *Description*: Create a 2-panel figure: approach time histogram and endpoint error distribution.
- `get_condition_label_from_peristim(peristim_path)` *Description*: Extract a descriptive condition label from PeriStim file. Returns labels like:
  - "Control" for control/baseline
  - "2 pulses @ 400Hz" for stim conditions
- `get_condition_label_from_metadata(br_idx, condition_type)` *Description*: Get condition label from metadata CSV. Returns labels like:
  - "Control (BR 4)" for control/baseline
  - "40p @ 400Hz (BR 5)" for stim conditions
- `create_endpoint_error_by_condition_svg(df, output_dir, condition_labels, summary_br_list)` *Description*: Create a publication-quality SVG figure of endpoint error by condition.
- `build_condition_labels(combined_df, peri_roots)` *Description*: Build a mapping from (br_idx, condition_type) to descriptive labels. Returns: dict: {(br_idx, condition_type): "label string"}
- `main()` *Description*: Main entry point.

`analyze_stim_conditions.py`

Path: scripts/nikita_scripts/random_row_generation_and_visualization/analyze_stim_conditions.py

Description: Validates the distribution, randomness, and overlap of stimulation channel combinations.

Functions:

- `parse_channel_string(chan_str)` *Description*: Parses "1-2 5 10-12" into [1, 2, 5, 10, 11, 12]
- `main()`

`generate_random_stim_conditions.py`

Path: scripts/nikita_scripts/random_row_generation_and_visualization/generate_random_stim_conditions.py

Description: Generates randomized combinations of cathodic/anodic stimulation channels.

Functions:

- `format_channels_compact(channels)` *Description*: Format a list of sorted integers into a space-separated string with ranges. E.g. [1, 2, 5, 6, 7] -> "1-2 5-7"
- `main()`

`plot_nprw_dist.py`

Path: scripts/nikita_scripts/random_row_generation_and_visualization/plot_nprw_dist.py

Description: Plots the anatomical distribution of Neuropixels rows targeted during a stimulation run.

`make_UA_mf_template.ipynb`

Path: scripts/nikita_scripts/ua_mf_template_and_comparison/make_UA_mf_template.ipynb

Description: Notebook for creating Utah Array matched-filter templates interactively.

`make_UA_mf_template.py`

Path: scripts/nikita_scripts/ua_mf_template_and_comparison/make_UA_mf_template.py

Description: Creates Utah Array matched-filter spike templates using SpikeInterface GUI tools.

Functions:

- `main()`

`mf_vs_threshold_comparison.ipynb`

Path: scripts/nikita_scripts/ua_mf_template_and_comparison/mf_vs_threshold_comparison.ipynb

Description: Notebook comparing SpikeInterface matched filtering detection with threshold-based detection.

Functions:

- `extract_waveforms(trace, peak_indices, n_before, n_after)` *Description*: Extract waveforms around peak indices.
- `compute_correlation(waveform, template)` *Description*: Compute normalized correlation between waveform and template.
- `plot_zoom_window(center_ms, window_ms)` *Description*: Plot a zoomed view around a specific time.

`mf_vs_threshold_comparison_consistent_a.py`

Path: scripts/nikita_scripts/ua_mf_template_and_comparison/mf_vs_threshold_comparison_consistent_a.py

Description: Compares spike detection counts and timestamps between Matched Filter and simple voltage thresholding.

`mf_vs_threshold_comparison_consistent_ab.ipynb`

Path:
scripts/nikita_scripts/ua_mf_template_and_comparison/mf_vs_threshold_comparison_consistent_ab.ipynb

Description: Notebook comparing SpikeInterface matched filtering detection with thresholding on dual-channel datasets.

Functions:

- `extract_waveforms(trace, peak_indices, n_before, n_after)` *Description*: Extract waveforms around peak indices.
- `compute_correlation(waveform, template_aligned)` *Description*: Compute normalized correlation between waveform and template.

Bryan's Scripts

Scripts created by Bryan (`scripts/bryan_scripts/`):

`RSA_plots.ipynb`

Path: scripts/bryan_scripts/RSA_consistency/RSA_plots.ipynb

Description: Implements Representational Similarity Analysis (RSA) on neural population vectors across conditions.

Functions:

- `_plot_rdm_conds(corr_mat, block_sizes, block_labels, title, out_svg, ax, vmin, vmax, show)` *Description*: Simple plot function to plot corr_mat (nTrial x nTrial) With ticks at block boundaries and labels at centers. Each block corresponds to one condition block (one NPZ file)
- `_plot_rdm_trial_index(corr_mat, title, out_svg, ax, vmin, vmax, tick_every, tick_start)` *Description*: Plot corr_mat with trial index ticks (for control only)
- `get_baseline_port(fname)` *Description*: Extract baseline port from file such as, baseline__NRR_RW011_Depth_43.0_port_B_target_A.npz Returns: Port A or B ("A" / "B" or None)
- `_get_data_from_peristim_npz(npz_path, source, corr_win_ms, channels, time_as_features, min_valid_features)` *Description*: Get feature matrix (X) from npz file (one block of trials) for one condition Outputs: X : (nTrials_kept, nFeatures) Each row is one trial labels : list[str] Trial labels for debugging; aligned with X rows info : dict Metadata including condition, target, is_baseline Npz contains:
  - NPRW_rates_zeroed or UA_rates_zeroed : (nTrials, nChannels, T)
  - NPRW_rel_t or UA_rel_t (T,) If time_as_features=False (default): # TODO fix this features = mean rate in [w0,w1] per channel => (n_trials, n_channels) If time_as_features=True: features = flatten (channels x timebins_in_window) => (n_trials, n_channels*T_sel)

- `_save_rsa_mat(out_mat, corr_mat, target, source, corr_win_ms, block_sizes, block_labels, X_shape)` *Description*: Save RSA matrix to a .mat file Saves: corr_mat - (nTrials, nTrials) meta - dictionary with target / source (UA or NPRW) / window and condition labels
- `run_trial_rsa_from_peristim(source, corr_win_ms)` *Description*: Compute and plot RSA per target cond_order sets the order in which the blocks are sorted Blocks sorted by (target, cond_rank, filename) If nothing is passed then (target, filename)
- `run_trial_rsa_control_only_from_peristim(source, corr_win_ms)` *Description*: Control-only RSA:
  - filter to blocks whose condition is in control_conds (default: just Baseline)
  - stack within each target
  - z-score within target (if enabled)
  - drop invalid rows
  - corrcoef(X)
  - plot with trial index ticks
- `run_per_channel_rsa_from_peristim(source, corr_win_ms, cond_label_extras, skip_conds, channels, keep_baseline_port, vmin, vmax)`

`RSA_poststim_grouped_up.ipynb`

Path: scripts/bryan_scripts/RSA_consistency/RSA_poststim_grouped_up.ipynb

Description: RSA analyzing population vector patterns in the post-stimulation epoch.

Functions:

- `_plot_rdm_with_block_ticks(SIM, block_sizes, block_labels, title, out_svg, ax, vmin, vmax)` *Description*: Plot *similarity* matrix (correlation r in [-1, 1]) with:
  - small ticks BETWEEN conditions at block boundaries (no labels)
  - text labels centered on each block (no ticks at the centers), mirrored on two axes. If ax is None, creates its own figure and handles saving/showing. If ax is provided, draws into that axes and leaves saving/showing to caller.
- `_trial_features_from_peristim_npz(npz_path, source, corr_win_ms, use_kinematics, channels)` *Description*: Build per-trial feature vectors from a peri-stim NPZ. Neural mode (use_kinematics=False):
  - NPRW_rates_zeroed / UA_rates_zeroed : (n_trials, n_channels, T)
  - NPRW_rel_t / UA_rel_t : (T,) Kinematics mode (use_kinematics=True):
  - beh_cam0_segs (and optionally beh_cam1_segs) : (n_rows, n_trials, T, ...)
  - beh_cam0_names : (n_rows,) Rows whose name is 'start', 'ta', or 'tb' are dropped.
  - Uses NPRW_rel_t / UA_rel_t for the time axis; CORR_WIN_MS applies to time.
  - Cam0 and Cam1 features are concatenated per trial.
- `_baseline_port_from_name(fname)` *Description*: Extract baseline port from filename like: baseline__NRR_RW011_Depth_43.0_port_B_target_A
- `run_trial_rsa_from_peristim(source, corr_win_ms, cond_label_extras, use_kinematics, vmin, vmax, skip_conds, baseline_port, channels, cond_order, average_within_condition)` *Description*: source:
  - "NPRW" -> neural, NPRW rates
  - "UA" -> neural, UA rates

    - "Kinematics" (or anything starting with "kin") -> kinematics mode average_within_condition:
    - If True, average all trials within each condition to produce one vector per condition before computing the RSM. This reduces the matrix from N_trials x N_trials to N_conditions x N_conditions.

`W_inference_ridge.ipynb`

Path: scripts/bryan_scripts/W_inference/W_inference_ridge.ipynb

Description: Notebook using Ridge regression to infer synaptic connectivity matrices.

Functions:

- `sequential_W_error(stim_select, perturb, rank, beta, sigma2, start)`
- `sequential_lowrank_reconst(stim, perturb, train_idx, test_idx, rank, beta, start)`

`W_inference_robust_LR.ipynb`

Path: scripts/bryan_scripts/W_inference/W_inference_robust_LR.ipynb

Description: Notebook using robust linear regression for connectivity matrix inference.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`

`RRR_NPRW_UA.ipynb`

Path: scripts/bryan_scripts/connectivity/RRR_NPRW_UA.ipynb

Description: Jupyter notebook implementing Reduced Rank Regression (RRR) to study functional connectivity between the Neuropixels (NPRW) and Utah Array (UA) populations.

Functions:

- `mask_stim_periods_and_shift(NPRW_rate, NPRW_t, UA_rate, UA_t, stim_ms_abs, delay_ms, stim_length)` *Description*: Mask [stim, stim+stim_length] and [stim+delay, stim+delay+stim_length] on each array using its own timebase, then return NPRW(t) aligned with UA(t+delay).

`PoisLDS_NPRW_combined.ipynb`

Path: scripts/bryan_scripts/dynamics/PoisLDS_NPRW_combined.ipynb

Description: Implements Poisson Latent Dynamical Systems (PLDS) models to extract latent state trajectories from Neuropixels populations.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`

- `make_params(C, d, A, B)`
- `loss_fn(theta, observations, inputs)` *Description*: Compute the negative log-likelihood using EKF like integrals. This is the Conditional Marginal Gaussian Smoother (CMGS) objective, which is an approximation to the true marginal log-likelihood. Returns: Negative log-likelihood + penalty for stability
- `step(theta, opt_state, observations, inputs)`
- `plot_latent_phase_portrait_shared(z_control, z_stim, centers_ms_control, centers_ms_stim, alpha_trials, stim_z1_offset)`
- `plot_mean_latents_and_psth(z_control, z_stim, psth_control, psth_stim, edges_ms_control, edges_ms_stim, centers_ms_control, centers_ms_stim, ir_idx, disp_states, vmin, vmax)`

`PoisLDS_NPRW_combined_all.ipynb`

Path: scripts/bryan_scripts/dynamics/PoisLDS_NPRW_combined_all.ipynb

Description: Expanded PLDS models applied to combined sessions/conditions.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`
- `make_params(C, d, A, B, mu0)`
- `loss_fn(theta, observations, inputs)` *Description*: Compute the negative log-likelihood using EKF like integrals. This is the Conditional Marginal Gaussian Smoother (CMGS) objective, which is an approximation to the true marginal log-likelihood. Returns: Negative log-likelihood + penalty for stability
- `step_batch(theta, opt_state, obs_batch, inp_batch)`
- `plot_latent_phase_portrait_sessions(z_by_session, centers_ms_by_session, control_indices, alpha_trials, plot_sessions, epoch, pre_trim_bins, post_trim_bins, show_trials, show_markers, show_pre_post, x_range, y_range, z_range)`
- `plot_mean_latents_and_psth(z_control, z_stim, psth_control, psth_stim, edges_ms_control, edges_ms_stim, centers_ms_control, centers_ms_stim, ir_idx, disp_states, vmin, vmax)`
- `pad_bins_to_edges(Z, centers_ms, edges_ms, tol)`

`PoisLDS_NPRW_control.ipynb`

Path: scripts/bryan_scripts/dynamics/PoisLDS_NPRW_control.ipynb

Description: PLDS modeling restricted to control (non-stimulated) trial periods.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`
- `make_params(C, d, A)`
- `loss_fn(theta, observations)` *Description*: Compute the negative log-likelihood using EKF like integrals. This is the Conditional Marginal Gaussian Smoother (CMGS) objective, which is an

approximation to the true marginal log-likelihood. Returns: Negative log-likelihood + penalty for stability

- `step(theta, opt_state, observations)` *Description*: Perform descent step using the CMGS negative log-likelihood as the loss function. Use Adam + autograd to compute gradients. Note: This is equivalent to fit_sgd for LinearGaussianSSM, but fit_sgd is not defined for GeneralizedGaussianSSM Returns: Updated parameters New optimizer state Current loss
- `plot_mean_latents_and_psth(means_rot, psth_plot, edges_ms, disp_states, vmin, vmax)`
- `fit_and_eval(all_observations, state_dim, key, n_epochs, lr)`
- `eval_loglik(params_fit, obs)`

- `plot_latent_phase_portrait_shared(z_control, z_stim, centers_ms_control, centers_ms_stim, alpha_trials, stim_z1_offset, time_window, show_markers, show_trials, show_z, highlight_segments)` *Description*: Parameters

  time_window : str "all" — plot every time point (default) "prestim" — plot only t < 0 ms "poststim" — plot only t >= 0 ms

- `export_rotating_gif(fig, output_path, n_frames, elevation, duration)` *Description*: Export a rotating Plotly 3D figure as a GIF. Parameters

  fig : go.Figure — your Plotly figure output_path: str — output .gif path n_frames : int — number of frames (72 = 5° per step) elevation : float — camera elevation angle in degrees duration : int — ms per frame (40ms = ~25 fps)
- `loss_fn(A, observations)`
- `step(A, opt_state, observations)`

`PoisLDS_NPRW_stim.ipynb`

Path: scripts/bryan_scripts/dynamics/PoisLDS_NPRW_stim.ipynb

Description: PLDS modeling analyzing Neuropixels population state trajectories during and after stimulation.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`
- `make_params(C, d, A)`
- `loss_fn(theta, observations)` *Description*: Compute the negative log-likelihood using EKF like integrals. This is the Conditional Marginal Gaussian Smoother (CMGS) objective, which is an approximation to the true marginal log-likelihood. Returns: Negative log-likelihood + penalty for stability
- `step(theta, opt_state, observations)` *Description*: Perform descent step using the CMGS negative log-likelihood as the loss function. Use Adam + autograd to compute gradients. Note: This is equivalent to fit_sgd for LinearGaussianSSM, but fit_sgd is not defined for GeneralizedGaussianSSM Returns: Updated parameters New optimizer state Current loss
- `plot_mean_latents_and_psth(means_rot, psth_plot, edges_ms, disp_states, vmin, vmax)`
- `plot_latent_phase_portrait(means_rot, edges_ms, alpha_trials)`

- `fit_and_eval(all_observations, state_dim, key, n_epochs, lr)`
- `eval_loglik(params_fit, obs)`

`PoisLDS_UA_control.ipynb`

Path: scripts/bryan_scripts/dynamics/PoisLDS_UA_control.ipynb

Description: PLDS modeling of Utah Array baseline motor cortex dynamics.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`
- `skew_to_rotation(S)` *Description*: A = expm(S - S.T) is always orthogonal with det=1 (rotation matrix)
- `make_params(C, d, A)`
- `loss_fn(theta, observations)` *Description*: Compute the negative log-likelihood using EKF like integrals. This is the Conditional Marginal Gaussian Smoother (CMGS) objective, which is an approximation to the true marginal log-likelihood. (E step of EM) Returns: Negative log-likelihood
- `step(theta, opt_state, observations)` *Description*: Perform optimization step using the CMGS negative log-likelihood as the loss function. (Equivalent to M step of EM, but using SGD). Note: This is equivalent to fit_sgd for LinearGaussianSSM, but fit_sgd is not defined for GeneralizedGaussianSSM Returns: Updated parameters New optimizer state Current loss
- `make_params(C, d, A)`
- `loss_fn(theta, observations)` *Description*: Compute the negative log-likelihood using EKF like integrals. This is the Conditional Marginal Gaussian Smoother (CMGS) objective, which is an approximation to the true marginal log-likelihood. (E step of EM) Returns: Negative log-likelihood
- `step(theta, opt_state, observations)` *Description*: Perform optimization step using the CMGS negative log-likelihood as the loss function. (Equivalent to M step of EM, but using SGD). Note: This is equivalent to fit_sgd for LinearGaussianSSM, but fit_sgd is not defined for GeneralizedGaussianSSM Returns: Updated parameters New optimizer state Current loss
- `plot_mean_latents_and_psth(means_rot, psth_plot, edges_ms, disp_states, vmin, vmax)`
- `plot_latent_phase_portrait(means_rot, edges_ms, alpha_trials)`
- `fit_and_eval(all_observations, state_dim, key, n_epochs, lr)`
- `eval_loglik(params_fit, obs)`

`jax_practice.ipynb`

Path: scripts/bryan_scripts/dynamics/jax_practice.ipynb

Description: Interactive notebook for learning and testing JAX code.

Functions:

- `make_params(C, d)`
- `plot_emissions_poisson(states, data)` *Description*: Plot the emissions of a Poisson :DS
- `loss_fn(theta, observations)` *Description*: Compute the negative log-likelihood using EKF like integrals. This is the Conditional Marginal Gaussian Smoother (CMGS) objective, which is an approximation to the true marginal log-likelihood. (E step of EM) Returns: Negative log-likelihood

- `step(theta, opt_state, observations)` *Description*: Perform optimization step using the CMGS negative log-likelihood as the loss function. (Equivalent to M step of EM, but using SGD). Note: This is equivalent to fit_sgd for LinearGaussianSSM, but fit_sgd is not defined for GeneralizedGaussianSSM Returns: Updated parameters New optimizer state Current loss
- `align_trajectories(ref, target)`
- `plot_observations_poisson(states, data, posts_true, posts_init, posts_fit, trial, use_procrustes)`

`PCA_analysis.ipynb`

Path: scripts/bryan_scripts/other/PCA_analysis.ipynb

Description: Dimensionality reduction using PCA to inspect population state spaces.

`nwb_test.ipynb`

Path: scripts/bryan_scripts/other/nwb_test.ipynb

Description: Tests conversion and structure verification of Neurodata Without Borders (NWB) formats.

`plot_HPF_traces_with_peaks.ipynb`

Path: scripts/bryan_scripts/quick_plots/plot_HPF_traces_with_peaks.ipynb

Description: Notebook to overlay detected spikes on high-pass filtered neural voltage traces.

Functions:

- `_parse_SI_peaks(peaks)`
- `_parse_SI_peaks(peaks)`
- `_parse_SI_peaks(peaks)`

`plot_UA_for_liz_dale.ipynb`

Path: scripts/bryan_scripts/quick_plots/plot_UA_for_liz_dale.ipynb

Description: Generates specialized Utah Array PSTH figures for collaborators.

Functions:

- `_parse_SI_peaks(peaks)`

`plot_aux_channels.ipynb`

Path: scripts/bryan_scripts/quick_plots/plot_aux_channels.ipynb

Description: Notebook to quickly plot raw analog auxiliary channels (sync waves, touchscreen signals).

Functions:

- `load_aux(npz_path)`

- `plot_signal(sig, fs, title, channel, start, end, max_points)`

`plot_fr_trials.ipynb`

Path: scripts/bryan_scripts/quick_plots/plot_fr_trials.ipynb

Description: Notebook to visualize trial-by-trial firing rate modulations.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`
- `pad_bins_to_edges(Z, centers_ms, edges_ms, tol)`

`plot_ns6_file.ipynb`

Path: scripts/bryan_scripts/quick_plots/plot_ns6_file.ipynb

Description: Notebook for inspecting raw 30kHz Blackrock NS6 recordings.

Functions:

- `_unit_to_kohm(val, unit)`
- `_read_text_loose(p)`
- `load_impedances_from_textedit_dump(path_like)` *Description*: Parse lines like 'elec1-5 201 kOhm' from a loose text dump. Returns { Elec#: impedance_kΩ }.
- `make_ua_keep_mask_from_imp(ua_ids_1based, imp_by_elec, max_kohm)` *Description*: Build a boolean mask of UA rows to keep. Policy:
  - If an electrode has a measured impedance <= max_kohm, keep it.
  - If impedance is > max_kohm, drop it.
  - If there is NO impedance entry for that electrode, drop it.

`plot_traces.ipynb`

Path: scripts/bryan_scripts/quick_plots/plot_traces.ipynb

Description: Simple visualizer for neural signal traces across channels.

Functions:

- `_anchor_from_shifts_row(row)` *Description*: From a shifts row, return (anchor_sample_intan, fs_intan). If anchor_sample is missing, derive from anchor_ms.

`plot_walking_ns6.ipynb`

Path: scripts/bryan_scripts/quick_plots/plot_walking_ns6.ipynb

Description: Notebook analyzing neural signals recorded during walking/locomotion.

Functions:

- `_dedup_peaks(peaks, fs, dedup_ms, max_cluster_ms, ch_field, samp_field, seg_field, amp_field)` *Description*: Deduplicate peaks per (segment, channel), keeping strongest within short windows
- `_compute_peak_times_per_seg(peaks, recording, fs, n_seg, samp_field, seg_field)`
- `_bin_counts_and_masks_artrmv(peaks, seg_n_samps, n_ch, n_seg, fs, bin_ms, blank_windows_samples, ch_field, samp_field, seg_field)`
- `_gauss_kernel_normalized(sigma_bins, radius_mult)`
- `_smooth_counts_with_blanks(counts_cat, blank_mask, sigma_ms, bin_ms)`
- `threshold_mua_rates(recording, detect_threshold, peak_sign, bin_ms, sigma_ms, n_jobs, blank_windows_samples)` *Description*: Threshold-crossing MUA, binned, Gaussian-smoothed firing rates. Returns

  rate_hz : (n_channels, n_bins) t_ms : (n_bins,) # bin-center times in ms (concatenated across segments) counts : (n_channels, n_bins) peaks : np.ndarray # structured array from SpikeInterface peak_t_ms : (n_peaks,) Global peak times in ms (segment offsets applied)
- `_unit_to_kohm(val, unit)`
- `_read_text_loose(p)`
- `load_impedances_from_textedit_dump(path_like)` *Description*: Parse lines like 'elec1-5 201 kOhm' from a loose text dump. Returns { Elec#: impedance_kΩ }.
- `make_ua_keep_mask_from_imp(ua_ids_1based, imp_by_elec, max_kohm)` *Description*: Build a boolean mask of UA rows to keep. Policy:
  - If an electrode has a measured impedance <= max_kohm, keep it.
  - If impedance is > max_kohm, drop it.
  - If there is NO impedance entry for that electrode, drop it.

`plot_peth_mf.ipynb`

Path: scripts/bryan_scripts/spike_detection/plot_peth_mf.ipynb

Description: Visualizes peri-event time histograms (PETH) using matched-filter sorted spikes.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`
- `peri_event_rates_from_peaks_ms(peaks_ms, event_times_ms, win_ms, bin_ms, sigma_ms)` *Description*: Compute peri-event firing rates from spike times (ms). Args

  peaks_ms: dict {unit_id: spike_times_ms} event_times_ms: (n_events,) array of event times in ms (same timebase as peaks_ms) win_ms: (start_ms, end_ms) peri-event window relative to each event bin_ms: bin width in ms sigma_ms: Gaussian smoothing sigma in ms (0 disables smoothing) Returns

rates_hz: (n_events, n_units, n_bins) smoothed peri-event rates in Hz centers_ms: (n_bins,) bin centers in ms relative to event unit_ids: list of unit_ids in the same order as axis=1

- `_parse_SI_peaks(peaks, n_channels)`
- `pad_bins_to_edges(Z, centers_ms, edges_ms, tol)`
- `_parse_SI_peaks(peaks, n_channels)`
- `pad_bins_to_edges(Z, centers_ms, edges_ms, tol)`
- `_parse_SI_peaks(peaks, n_channels)`
- `pad_bins_to_edges(Z, centers_ms, edges_ms, tol)`

`spikesort_control.ipynb`

Path: scripts/bryan_scripts/spike_detection/spikesort_control.ipynb

Description: Spike sorting workflows using SpikeInterface on control sessions.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`
- `peri_event_rates_from_peaks_ms(peaks_ms, event_times_ms, win_ms, bin_ms, sigma_ms)` *Description*: Compute peri-event firing rates from spike times (ms). Args

  peaks_ms: dict {unit_id: spike_times_ms} event_times_ms: (n_events,) array of event times in ms (same timebase as peaks_ms) win_ms: (start_ms, end_ms) peri-event window relative to each event bin_ms: bin width in ms sigma_ms: Gaussian smoothing sigma in ms (0 disables smoothing) Returns

  rates_hz: (n_events, n_units, n_bins) smoothed peri-event rates in Hz centers_ms: (n_bins,) bin centers in ms relative to event unit_ids: list of unit_ids in the same order as axis=1
- `pad_bins_to_edges(Z, centers_ms, edges_ms, tol)`
- `peaks_ms_from_sorting(sorting, fs, segment_index)`

`spikesort_stim.ipynb`

Path: scripts/bryan_scripts/spike_detection/spikesort_stim.ipynb

Description: Spike sorting workflows on stimulation sessions, evaluating artifact blanking.

Functions:

- `_parse_SI_peaks(peaks, n_channels)`
- `peri_event_rates_from_peaks_ms(peaks_ms, event_times_ms, win_ms, bin_ms, sigma_ms)` *Description*: Compute peri-event firing rates from spike times (ms). Args

peaks_ms: dict {unit_id: spike_times_ms} event_times_ms: (n_events,) array of event times in ms (same timebase as peaks_ms) win_ms: (start_ms, end_ms) peri-event window relative to each event bin_ms: bin width in ms sigma_ms: Gaussian smoothing sigma in ms (0 disables smoothing) Returns

rates_hz: (n_events, n_units, n_bins) smoothed peri-event rates in Hz centers_ms: (n_bins,) bin centers in ms relative to event unit_ids: list of unit_ids in the same order as axis=1

- `peaks_ms_from_sorting(sorting, fs, segment_index)`

Metadata & Configuration Files in `scripts/`**

- `electrode_port_mapping.csv`: scripts/electrode_port_mapping.csv
- `physical_grids_layout.csv`: scripts/physical_grids_layout.csv